

Problem List

1154. Day of the Year

Solved

Easy

Topics

Companies

Hint

Given a string `date` representing a **Gregorian calendar** date formatted as `YYYY-MM-DD`, return the *day number of the year*.

Example 1:

Input: `date = "2019-01-09"`
Output: 9
Explanation: Given date is the 9th day of the year in 2019.

Example 2:

Input: `date = "2019-02-10"`
Output: 41

Constraints:

- `date.length == 10`
- `date[4] == date[7] == '-',` and all other `date[i]`'s are digits
- `date` represents a calendar date between Jan 1st, 1900 and Dec 31st, 2019.

518 36 5 Online

Code

Java

Auto

```
1 class Solution {
2     public int dayOfYear(String date) {
3
4         int year = Integer.parseInt(date.substring(0, 4));
5         int month = Integer.parseInt(date.substring(5, 7));
6         int day = Integer.parseInt(date.substring(8, 10));
7
8         int[] days = {
9             31, 28, 31, 30, 31, 30,
10            31, 31, 30, 31, 30, 31
11        };
12
13        // Leap year
14        if (isLeapYear(year)) {
15            days[1] = 29;
16        }
17
18        int result = day;
19
20        for (int i = 0; i < month - 1; i++) {
21            result += days[i];
22        }
23
24        return result;
25    }
26 }
```

Saved Ln 3, Col 7

Testcase Test Result

Accepted Runtime: 0 ms

Problem List

1185. Day of the Week

Solved

Easy

Topics

Companies

Hint

Given a date, return the corresponding day of the week for that date.

The input is given as three integers representing the **day**, **month** and **year** respectively.

Return the answer as one of the following values ("Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday").

Note: January 1, 1971 was a Friday.

Example 1:

Input: `day = 31, month = 8, year = 2019`
Output: "Saturday"

Example 2:

Input: `day = 18, month = 7, year = 1999`
Output: "Sunday"

Example 3:

Input: `day = 15, month = 8, year = 1993`
Output: "Sunday"

Code

Java

Auto

```
1 import java.time.LocalDate;
2
3 class Solution {
4     public String dayOfTheWeek(int day, int month, int year) {
5         LocalDate date = LocalDate.of(year, month, day);
6
7         return date.getDayOfWeek().toString().charAt(0)
8             + date.getDayOfWeek().toString().substring(1).toLowerCase();
9     }
10 }
```

Saved Ln 1, Col 1

Testcase Test Result

Accepted Runtime: 4 ms

Case 1 Case 2 Case 3

Input

day = 31

month = 8

Problem List

451. Sort Characters By Frequency

Medium

Topics

Companies

Given a string `s`, sort it in **decreasing order** based on the **frequency** of the characters. The **frequency** of a character is the number of times it appears in the string.

Return the *sorted string*. If there are multiple answers, return *any of them*.

Example 1:

Input: `s = "tree"`
Output: "eert"
Explanation: 'e' appears twice while 'r' and 't' both appear once. So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

Example 2:

Input: `s = "cccaaa"`
Output: "aaaccc"
Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers. Note that "cacaca" is incorrect, as the same characters must be together.

Example 3:

Code

Java

Auto

```
1 class Solution {
2     public String frequencySort(String s) {
3
4         Map<Character, Integer> map = new HashMap<>();
5
6         for (char c : s.toCharArray()) {
7             map.put(c, map.getOrDefault(c, 0) + 1);
8         }
9         List<Character> chars = new ArrayList<>(map.keySet());
10
11        // Sort by frequency descending
12        chars.sort((a, b) -> map.get(b) - map.get(a));
13
14        // Build result
15        StringBuilder result = new StringBuilder();
16
17        for (char c : chars) {
18            int count = map.get(c);
19
20            for (int i = 0; i < count; i++) {
21                result.append(c);
22            }
23        }
24
25        return result.toString();
26    }
27 }
```

Saved Ln 9, Col 9

Testcase Test Result

Accepted Runtime: 1 ms

Problem List

DescriptionEditorialSolutionsSubmissions

179. Largest Number

MediumTopicsCompanies

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

Input: `nums = [10,2]`
Output: `"210"`

Example 2:

Input: `nums = [3,30,34,5,9]`
Output: `"9534330"`

Constraints:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 109`

9.6K22866 Online

Code

JavaAuto

```
1 import java.util.*;
2
3 class Solution {
4     public String largestNumber(int[] nums) {
5
6         String[] arr = new String[nums.length];
7
8         for (int i = 0; i < nums.length; i++) {
9             arr[i] = String.valueOf(nums[i]);
10        }
11
12        Arrays.sort(arr, (a, b) -> (b + a).compareTo(a + b));
13
14        // If all numbers are 0
15        if (arr[0].equals("0")) {
16            return "0";
17        }
18
19        StringBuilder result = new StringBuilder();
20
21        for (String s : arr) {
22            result.append(s);
23        }
24
25        return result.toString();
26    }
27 }
```

SavedLn 1, Col 1

TestcaseTest Result

HackerRank |
 [Prepare](#) |
 [Java](#) |
 [Data Structures](#) |
 [Java Sort](#)
[Exit Full Screen View](#)

Problem

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two students have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

Input Format

The first line of input contains an integer N , representing the total number of students. The next N lines contain a list of student information in the following structure:

```
ID Name CGPA
```

Constraints

- $2 \leq N \leq 1000$
- $0 \leq ID \leq 100000$
- $5 \leq |Name| \leq 30$
- $0 \leq CGPA \leq 4.00$

The name contains only lowercase English letters. The ID contains only integer numbers without leading zeros. The $CGPA$ will contain, at most, 2 digits after the decimal point.

Output Format

After rearranging the students according to the above rules, print the first name of each

```

8 public Student(int id, String name, double cgpa) {
9     this.id = id;
10    this.name = name;
11    this.cgpa = cgpa;
12 }
13
14 public int getId() {
15     return id;
16 }
17
18 public String getName() {
19     return name;
20 }
21
22 public double getGpa() {
23     return cgpa;
24 }
25 }
```

Line: 56 Col: 1

☐ Test against custom input

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0(run it)

HackerRank

PrepareJavaData StructuresJava Comparator

Problem

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The Player class is provided for you in your editor. It has 2 fields: a *name* String and a *score* integer.

Given an array of *n* Player objects, write a comparator that sorts them in order of decreasing score; if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from stdin is handled by the locked stub code in the Solution class.

The first line contains an integer, *n*, denoting the number of players.

Each of the *n* subsequent lines contains a player's *name* and *score*, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- 2 players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in Solution

Change ThemeLanguageJava 8

```
1 import java.util.*;  
2 class Checker implements Comparator<Player> {  
3     @Override  
4     public int compare(Player p1, Player p2) {  
5         // Higher score comes first  
6         if (p1.score != p2.score) {  
7             return Integer.compare(p2.score, p1.score);  
8         }  
9  
10        // If scores are equal, alphabetical order by name  
11        return p1.name.compareTo(p2.name);  
12    }  
13 }  
14 > class Player{ ...
```

HackerRank |
 [Prepare](#) |
 [Java](#) |
 [Data Structures](#) |
 [Java ArrayList](#)

Sometimes it's better to use dynamic size arrays. Java's [ArrayList](#) can provide you this feature. Try to solve this problem using [ArrayList](#).

You are given n lines. In each line there are zero or more integers. You need to answer a few queries where you need to tell the number located in y^{th} position of x^{th} line.

Take your input from System.in.

Input Format

The first line has an integer n . In each of the next n lines there will be an integer d denoting number of integers on that line and then there will be d space-separated integers. In the next line there will be an integer q denoting number of queries. Each query will consist of two integers x and y .

Constraints

- $1 \leq n \leq 20000$
- $0 \leq d \leq 50000$
- $1 \leq q \leq 1000$
- $1 \leq x \leq n$

Each number will fit in signed integer.
Total number of integers in n lines will not cross 10^5 .

Output Format

In each line, output the number located in y^{th} position of x^{th} line. If there is no such position, just print "ERROR"

Sample Input

```

5
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25

```

Change Theme
Language
Java 8

```

1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         int n = sc.nextInt();
10
11         ArrayList<ArrayList<Integer>> list = new ArrayList<>();
12
13         // Read the lines
14         for (int i = 0; i < n; i++) {
15             int d = sc.nextInt();
16
17             ArrayList<Integer> numbers = new ArrayList<>();
18
19             for (int j = 0; j < d; j++) {
20                 numbers.add(sc.nextInt());
21             }
22             list.add(numbers);
23         }
24     }
25 }

```

Line: 47 Col: 1

☐ Test against custom input

Problem

In computer science, a priority queue is an abstract data type which is like a regular queue, but where additionally each element has a "priority" associated with it. In a priority queue, an element with high priority is served before an element with low priority. - [Wikipedia](#)

In this problem we will test your knowledge on [Java Priority Queue](#).

There are a number of students in a school who wait to be served. Two types of events, ENTER and SERVED, can take place which are described below.

- ENTER: A student with some priority enters the queue to be served.
- SERVED: The student with the highest priority is served (removed) from the queue.

A unique id is assigned to each student entering the queue. The queue serves the students based on the following criteria (priority criteria):

1. The student having the highest Cumulative Grade Point Average (CGPA) is served first.
2. Any students having the same CGPA will be served by name in ascending case-sensitive alphabetical order.
3. Any students having the same CGPA and name will be served in ascending order of the id.

Create the following two classes:

- The Student class should implement:
 - The constructor `Student(int id, String name, double cgpa)`.
 - The method `int getID()` to return the id of the student.
 - The method `String getName()` to return the name of the student.
 - The method `double getCGPA()` to return the CGPA of the student.

Discussions

Leaderboard

Submissions

```
Change Theme Language Java 8
1 > import java.util.ArrayList;
4 import java.util.PriorityQueue;
5 import java.util.Comparator;
6
7 class Student {
8     private int id;
9     private String name;
10    private double cgpa;
11
12    public Student(int id, String name, double cgpa) {
13        this.id = id;
14        this.name = name;
15        this.cgpa = cgpa;
16    }
17
18    public int getID() {
19        return id;
20    }
21
22    public String getName() {
23        return name;
24    }
25
26    public double getCGPA() {
27        return cgpa;
28    }
29 }
```

Line: 4 Col: 1

☐ Test against custom input